Machine Learning for Manufacturing Engineering Optimisation

Isaac Batho

Student Number: 11013249

Course: BEng Aerospace Engineering, Year 3

Unit: AERO31030 Individual Project (2025–26)

Supervisor: Dr. Moray Kidd

Department of Mechanical and Aerospace Engineering

University of Manchester School of Engineering 2025-26

March 26, 2026

Abstract

Optimisation is an integral component of designing manufacturing systems and processes to get the most effective output. Machine learning promises to improve the optimisation process. Optimization can significantly increase utilization rates, reduce costs, shorten production cycles, and improve the quality of the final product. Improvements in machine learning allow us to handle more data and be more versitile which can be applied to optimisation and all manufacturing processes to be developed faster.

Lay Summary

Manufacturing components can require multiple different processes using a range of machines. Each of these processes will have their individual times. Planning when to use each tool work on a certain job is a complex task with differnt processing times needing to be accomodated but this becomes even more complex when processing orders with differnt processes.

Contents

Abstract **i**

Lay Summary **ii**

1 Introduction **1**

2 Aims and Objectives **2**

3 Literature Review **4**

3.1 Research methodology 4

3.2 The Need for Optimisation 4

3.3 Conventional Optimisation Methods 4

3.4 Limitations of Conventional Optimisation Methods 8

3.5 Machine Learning for Optimisation 8

3.5.1 Conclusion 10

4 Methodology **11**

4.1 Machine Learning Methods 11

4.2 Tools 11
4.3 Case Studies 12
4.3.1 Simplified Flexible Job Shop 13
4.3.2 Simplified Flexible Job Shop with Buffers 15
4.3.3 Simplified Flexible Job Shop with Buffers Running Shifts 15
4.4 Transfer Function 15

5 Results **16**
5.1 Simplified Flexible Job Shop 16
5.2 Simplified Flexible Job Shop with Buffers 18
5.3 Simplified Flexible Job Shop with Buffers Running Shifts 18

6 Discussion **19**
6.0.1 Training Curves 19
6.0.2 Schedule Produced 19
6.1 Case Studies 19
6.1.1 Simplified Flexible Job Shop 19
6.1.2 Simplified Flexible Job Shop with Buffers 19
6.1.3 Simplified Flexible Job Shop with Buffers Running Shifts 19
6.2 Answers to Research Questions 19
6.2.1 What are the capabilities and limits of different machine learning methods, programming languages, libraries, and tools available for me to use? 19
6.2.2 What data and parameters need to be gathered, provided, and calculated for optimising manufacturing processes? 20
6.2.3 What programming language, machine learning method, libraries and other tools will be used in the project based on the data and parameters being used? 20
6.2.4 What are the capabilities and limits of this program developed? . . . 20
6.2.5 Is this method of optimisation as good as or better than existing methods and how should it be deployed? 20

7 Environmental, Societal, and Ethical Impact **21**

8 Conclusions **22**

References 22

A Appendix A: Project Planning **25**

B Appendix B: Risk Assessment **26**

C Additional Appendices **27**

List of Figures

3.1 Flow of the dynamic solving process for the DFFS (Tang et al. 2016) 7

3.2 The process of improved particle swarm optimization algorithm combined with reinforcement learning 10

4.1 A Factory Layout of order flows between FUs 12

5.1 Loss Curve 16

5.2 Reward Curve 17

5.3 Caption 17

5.4 Caption 17

List of Tables

5.1 Example Order Specification 16

Nomenclature

1 Introduction

The problem in optimising manufacturing processes is that for the most part each time there is a state change, the optimisation algorithm needs to be re-run. A state change refers to events such as new orders, machine breakdowns, delays. When such algorithms have to be re-run they take require high computational cost and time to be performed which means

2 Aims and Objectives

Objective 1

Research machine learning methods, programming languages, libraries, and tools available for me to use.

Research Question 1

What are the capabilities and limits of different machine learning methods, programming languages, libraries, and tools available for me to use?

Objective 2

Research current optimisation techniques used both in industry and being researched.

Research Question 2

What data and parameters need to be gathered, provided, and calculated for optimising manufacturing processes?

Objective 3

Review the research to identify the most appropriate methods, languages, libraries, and tools for the data and parameters used for optimisation.

Research Question 3

What programming language, machine learning method, libraries and other tools will be used in the project based on the data and parameters being used?

Objective 4

Develop a program that designs an optimised manufacturing process using machine learning.

Research Question 4

What are the capabilities and limits of this program developed?

Objective 5

Validate and recommend the methods developed and how they should be used for manufacturing optimisation.

Research Question 5

Is this method of optimisation as good as or better than existing methods and how should it be deployed?

3 Literature Review

3.1 Research methodology

saunders reseach onion

3.2 The Need for Optimisation

To effectively optimise any process no matter what the methods it is important to know what the aim of optimisation is. In addition, identifying which goals should be prioritized is important for the best performance for the chosen domain. Most manufacturers are profit driven and as such aim to reduce costs and increase revenue. Looking at the key performance indicators (KPIs) of large companies gives an indication what the goals of optimisation should be. Ahmad & Dhafr (2002) defines a KPI as "a number or value which can be compared against an internal target, or an external target 'benchmarking' to give an indication of performance". Looking at annual reports from 2024 and 2025 of 5 major engineering companies the main KPIs talked about are: revenue/sales, profits, return on investment, injury rates, and green house gas emissions (*Annual Financial Report 2024* (n.d.);*Babcock Annual Report and Financial Statements 2025* (n.d.))

3.3 Conventional Optimisation Methods

Heuristics

The simplest forms of optimisation are heuristics such as: Shortest Processing Time (SPT), prioritise the job with the shortest time; Longest Processing Time (LPT), prioritising the job with the longest time (Alhijawi & Awajan 2024); Longest Processing Time (LPT), prioritizes the job with the longest processing time; Earliest Due Date (EDD), A static rule that prioritizes

jobs based on their due date; First In First Out / First Come First Served (FIFO / FCFS), Processes jobs in the exact order they arrive; Minimum Slack Time (MST), Prioritizes jobs with the least amount of slack time (Ruiz & Maroto 2005). However, these a short sighted and only optimise for a single objective meaning they have limited use and aren't very effective on real factories. Ruiz & Maroto (2005) demonstrated the low performance of using these methods and pointed out their significant use in industry at the time which highlights the possibility to improve operations drastically with more effective methods. Further more, heuristics are fixed and as such can't accommodate flexible workshops which can use different possible routes or processes for production so inherently they won't always find the optimal solution as they can't consider different possibilities (Subramaniam n.d.)

Surface Response Methodology (RSM)

RSM was developed in the 1950s and provides optimal variables in the desired region of response. This region is where the output values which are being maximised or minimised co-coincide (Khuri & Mukhopadhyay 2010). This has the advantages of not needing many experiments and short cycles to optimise while also being very accurate (Song et al. 2024). However, modern computer software can determine the overlap of desired optimal values quicker, easier and with more precision.

Genetic Algorithm (GA)

GAs work by taking an initial population of possible solutions to the task being optimised. From this population a set of candidates are chosen using a variety of techniques (e.g. Stochastic, roulette,). The every solution in the population will have a fitness function which compares how the solution performs to the average of the population. Solutions with a higher fitness function have a higher probability of being selected. Those selected with then have their processes combined with other selected solutions to form new processes. These new solutions then either completely replace the last population or a part of the last population is removed and the new solutions added in. This process is then repeated for a set amount of iterations or until another stopping criterion is met Alhijawi & Awajan 2024.

GAs are good at searching for a wide variety of solutions without getting stuck in local solutions and can be very adaptable to different optimisation problems. In addition, they can

handle non-linear, non-convex optimisation problems, and dynamic environments.

The disadvantage of GAs is that it takes a long time to determine a solution and as such the parameters may need to be adjusted to improve the performance (Song et al. 2024). For frozen or slowly changing fields GAs work well but since they need to be re-optimised every time the plan is changed they do not work for dynamic environments such as when multiple orders are being placed over time and need scheduling quickly and reliably.

Particle Swarm Optimisation (PSO)

Random initial values are generated for the positions and velocity vectors of particles in a defined population size with each particle being given a fitness value based on this random initial position. Since this is the initial position it will considered the best fitness function for that particle and the next position is found using the initial velocity vector from the initial position. The best fitness function of all particles is known a the global best fitness function. To determine the next movement of each particle the vectors of velocity (inertia), from the particle position to the global best position, and from the current position to particle's best position. This is iterated to find the best position after a set amount of time given for optimisation. Weightings can be given to the different components deciding the new position depending on which factor is desired to be favoured: inertia, individual best position, or global best position.

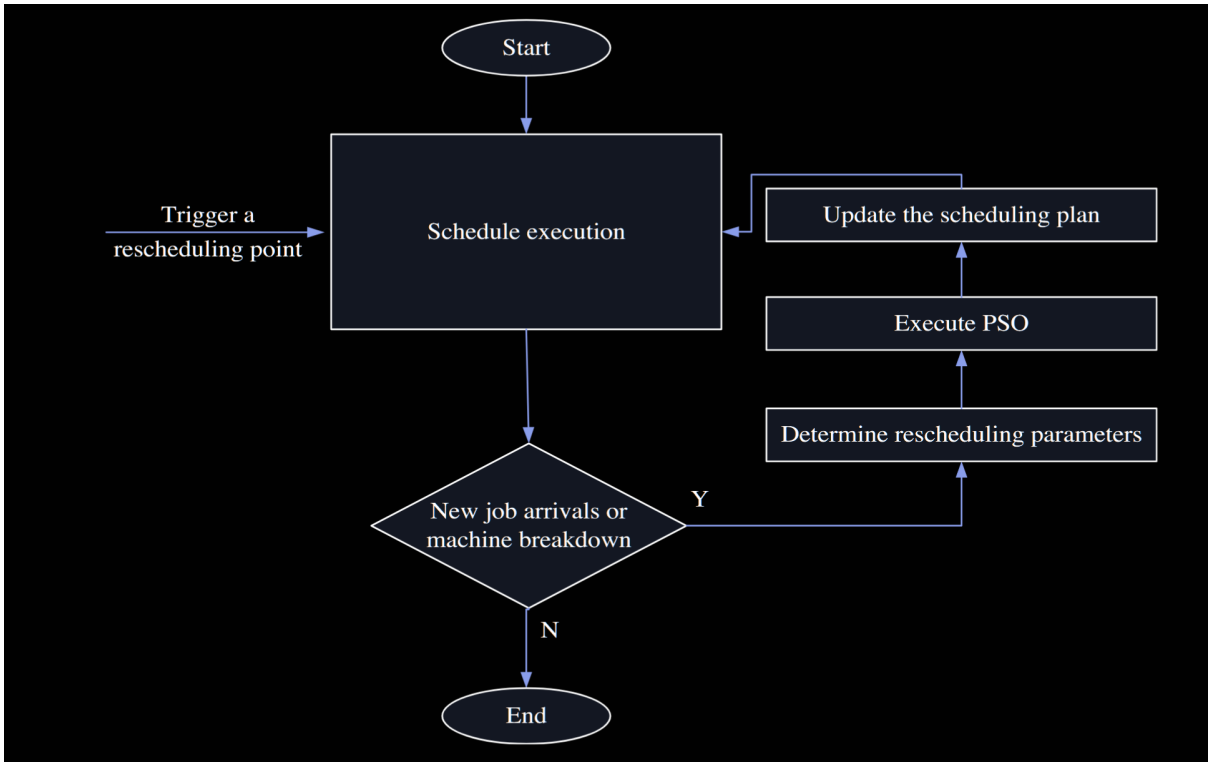


Figure 3.1: Flow of the dynamic solving process for the DFFS (Tang et al. 2016)

Figure 3.1 illustrates the rescheduling logic used in PSO when there's a new order, machine breakdown or other rescheduling trigger point.

Using particle swarm optimisation risks getting trapped in a local optimal solution in where the particles converge to one solution that appears to be optimal but there is actually a better solution that hasn't been explored. This can be mitigated by increasing the amount of particles used and time given for optimisation but this comes at the expense of being more computationally expensive (Tang et al. 2016).

Taguchi Method

Monte Carlo

Combination of Techniques

PSO techniques are often used in conjunction with a GA to provide a more comprehensive solution (Song et al. 2024)

3.4 Limitations of Conventional Optimisation Methods

The main limitation facing conventional optimisation methods is the need for rescheduling when there is a state change. Furthermore, the static nature of these algorithms means they can't react to unexpected events such as a machine breakdown. There are strategies to provide short term reactive rescheduling but these are either limited or unrestricted which Tang et al. (2016) found can cause instability resulting in useless plans. The Job Shop Scheduling Problem is an NP hard problem where the number of states increases rapidly, meaning the computational cost gets increasingly higher and takes longer. This has resulted in approximation methods to be used which do not provide optimal solutions (Meeran & Morshed 2012). Large computational costs often result in only a single solution provided which removes human insight the opportunity to look into multiple possibilities to identify different possibilities (Wang & Shan 2007).

3.5 Machine Learning for Optimisation

Machine learning (ML) techniques have been developing rapidly in recent years and are being adopted across industries and disciplines. A key ability of ML is pattern recognition which is being researched and used for optimisation particularly for similar cases where at least partial repeatability is useful. While conventional optimisation techniques can often struggle with optimising for multiple objectives ML is able to manage this with different methods but often by employing tensors to hold all the values. Moreover, it is possible to handle quantitative as well as qualitative data which can then be used either as variables in the process or as objectives (Prestwich 2024). These methods can help convert tacit domain knowledge of those who have learnt from being in the industry over time into factors that can be taken into account for optimisation. ML for the most part comes under either supervised, unsupervised, and reinforcement learning (Ragab et al. 2019).

Policy can be used on unseen examples provided the same action and observation space. Learns a general policy rather than specific. Can be used for stochastic environments when events aren't certain and can be ascribed possibilities providing robust and autonomous control.

However, to be useful and provide effective results, high amounts of data and computational power is needed making their use inaccessible to many (Song et al. 2024). Breif explainer on

3 ML concepts RL specifics will be covered in development

Generative Design

Klar et al. (2021) used reinforcement learning to plan factory layouts using a Double Deep Q Learning (DDQL). They modelled the factory floor as a grid and trained an agent to select the rotation and position of a Functional Unit (FU) along an automated guided vehicle (AGV) track. The model learned to plan the layout for reducing transport time and as such placed the FUs close together. This is a good example of optimising before operations even start and an effective solution was found in 8000 episodes.

However, the scalability and use of this was quite limited since only 4 FUs were trained for and doesn't appear to be able to be used on alternative factories with different FUs. In addition, only time was optimised for again limiting the effectiveness in the real world along with the lack of restrictions such as electricity and water supply and usage.

DDQL is usable for such a small scale optimisation but it takes long training times to provide effective results. For use with larger states which would be the case with more FUs and objectives DDQL would struggle to provide effective solutions or cost a lot to train (Sabbir M 2024).

Optimising Production Schedules

Gao et al. (2023) used reinforcement learning to aid PSO by adding it into the algorithm's optimisation process to change the weightings of different components: inertia, individual best position, or global best position. Based on the state of the particles the algorithm can then move between global exploration, local search, slow convergence, and fast convergence. The fitness functions of different iterations are compared to provide a reward function from which a Q-table is updated to be used for future decisions. The iterations are carried out until a termination criteria is met such as a maximum number of iterations or an acceptable error is reached. The logic for this process and the integration of RL into the algorithm can be seen bellow in Figure 3.2

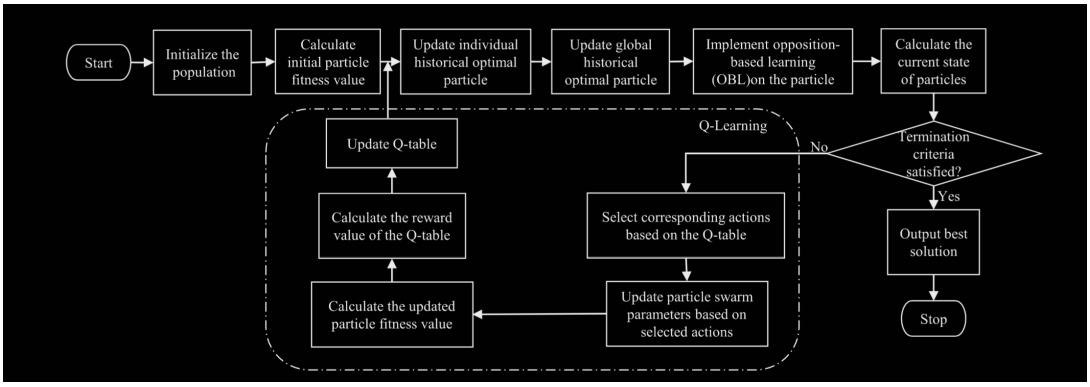


Figure 3.2: The process of improved particle swarm optimization algorithm combined with reinforcement learning

This Improved Particle Swarm Optimisation with Reinforcement Learning (IPSO-RL) was validated by comparing the results of testing to optimal values for PSO and GWO suggested by previous literature. With the same population size, maximum number of generations, and number of times algorithm was tested in the same case, IPSO-RL was found to have better convergence performance than other algorithms used for solving FJSP.

While improvements are clear this doesn't solve the main problem identified earlier as dynamic rescheduling isn't possible so still has to be re-optimised. The use of RL in this case if for an improvement on the current rather than a new abilities.

3.5.1 Conclusion

Wrap up main points learnt and why RL is used

4 Methodology

4.1 Machine Learning Methods

ML, at its core, is about training algorithms on data so they can learn patterns and relationships and then automatically produce useful predictions or decisions for new inputs. Given that there aren't publicly available data sets of schedules for workshops with times to work on each job at each FU with decisions made, synthetic data had to be created by simulating factory operations. Reinforcement learning is the most appropriate(only) machine learning technique for training an algorithm on these processes. It also has the additional benefit that it's trained by trial and error and not from past plans, which could mean learning to make less effective decisions.

4.2 Tools

Machine Learning Libraries

A widely used standard interface for training RL agents is Gymnasium which is an open-source python library originally created by OpenAI and now maintained by The Farama Foundation. Originally known as gym, the library allows both standard and custom environments to be created, RL algorithms to be trained and evaluated, and rollouts to be visualised.
Gym is compatible with a variety of different methods to train RL algorithms. Stable-Baselines3 is an open-source library that integrates easily with Gymnasium and was used in this work to train models with several available algorithms. In addition, custom algorithms can be implemented using other libraries such as PyTorch and TensorFlow.
To run simulations, SimPy was wrapped inside the Gymnasium environment for continuous time advancement, rather than a fixed time step per environment step.

Programming Languages

Python is the de facto industry standard for training machine learning models, largely because modern ML frameworks (e.g. TensorFlow, PyTorch) make their core functions, classes, and configuration options available through Python APIs. It's simple syntax, extensive scientific libraries, and large community make experimentation, collaboration, and deployment significantly easier than with most alternative languages. In addition, Python is a robust, general-purpose language with rich data processing capabilities and access to a vast ecosystem of libraries for tasks such as data cleaning, transformation, and integration, which further streamlines end-to-end ML workflows.

4.3 Case Studies

Job Shop Scheduling is a NP-hard problem; as manufacturing complexity grows, traditional manual scheduling becomes a bottleneck. Workshops with multiple **Functional Units (FUs)** and orders as illustrated in Figure **??** need a quick and effective method of optimising production to their priorities.

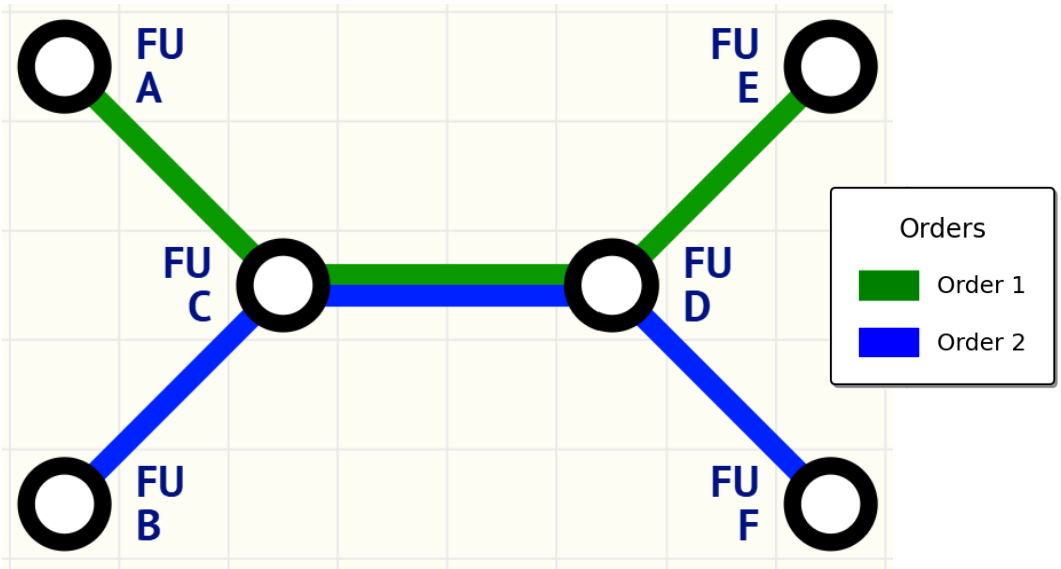


Figure 4.1: A Factory Layout of order flows between FUs

4.3.1 Simplified Flexible Job Shop

In the Flexible Job Shop Scheduling Problem (FJSP) discussed in the Literature Review assumptions are made to remove complexity in the real world. This makes modelling processes much more simple allowing for testing of different techniques.

The assumptions of this case study are:

- Continuous availability: every FU is considered to be operational at all times with machine breakdowns being neglected.
- Deterministic processing times: exact times for every operation is known prior to optimisation.
- Fixed process order: each job must follow it's predefined flow through the workshop spending the given time at each FU.

The differences from classical FJSP which demonstrates the handling of more complex real world scenarios are:

- Not all jobs are available at time zero: jobs are given a start time meaning they will begin at different points. Thus, the agent has to learn how to introduce other orders while already processing other jobs.

Observation Space

The observation space is what the agent state "sees". This is the data it is provided and for each state will have an associated action to take. These values are put into a tensor to be computed.
For every Order:

- Normalised jobs to do - $\frac{\text{Jobs To Do}}{\text{Total Jobs}}$
- Normalised jobs complete - $\frac{\text{Jobs Complete}}{\text{Total Jobs}}$
- Normalised time until due - $\frac{\text{Due Date} - \text{Current Time}}{\text{Due Date} - \text{Start Time}}$

- Binary signal for each FU to identify which one is the first FU in that order's route

For every FU:

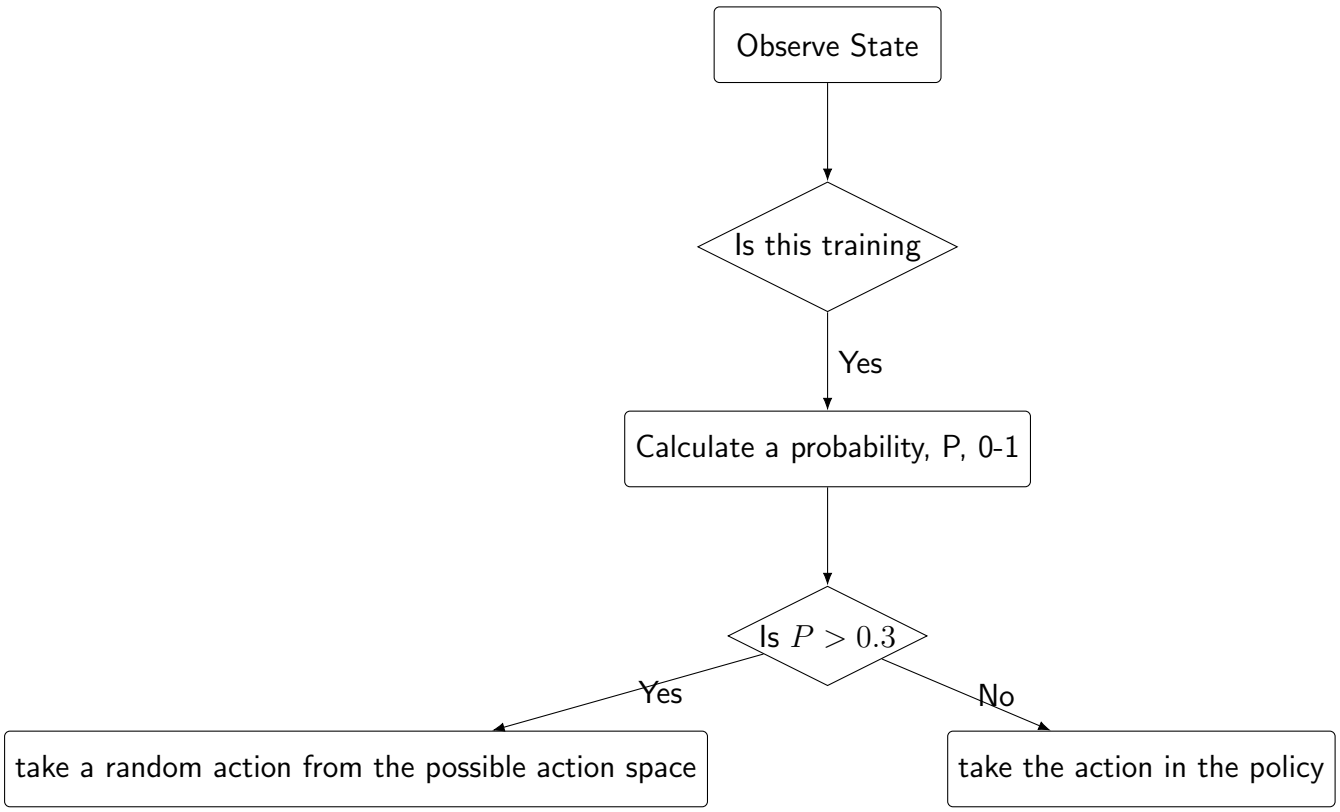
- Binary signal for busy or not
- Utilisation rate clipped between 0 and 1 - $\frac{\text{Working Time}}{\text{Total Time}}$
- Idle rate - $\frac{\text{T}}{}$

Action Space

The action each FU takes working from the last FUs in the order to the first. For every FU it can take an action of:

- 0 - do nothing
- 1 - move part to it's next FU if available
- n - start a job from order n-1

Transfer Function



4.3.2 Simplified Flexible Job Shop with Buffers

Assumptions

Transfer Function

4.3.3 Simplified Flexible Job Shop with Buffers Running Shifts

Assumptions

Transfer Function

4.4 Transfer Function

write pseudo code for logic illustration

Observation of system state given, for each FU: and each Order: and other info:

Is training?

Y: Prob of random action P

if P ¡ 0.3: random action taken

else: action given from policy

Reward calc from action

count increased

if over max count: terminate

if reached all orders complete: stop + big overall reward

Calc overall reward

N: action given from policy

new state found

5 Results

5.1 Simplified Flexible Job Shop

Inputs

Table 5.1: Example Order Specification

Order	Start Time	Due Date	Order Size	Route: Working Time
1	0	75	6	A:4-¿C:2-¿D:3-¿E:3
2	5	90	5	B:5-¿C:2-¿D:3-¿F:2

Training Curves

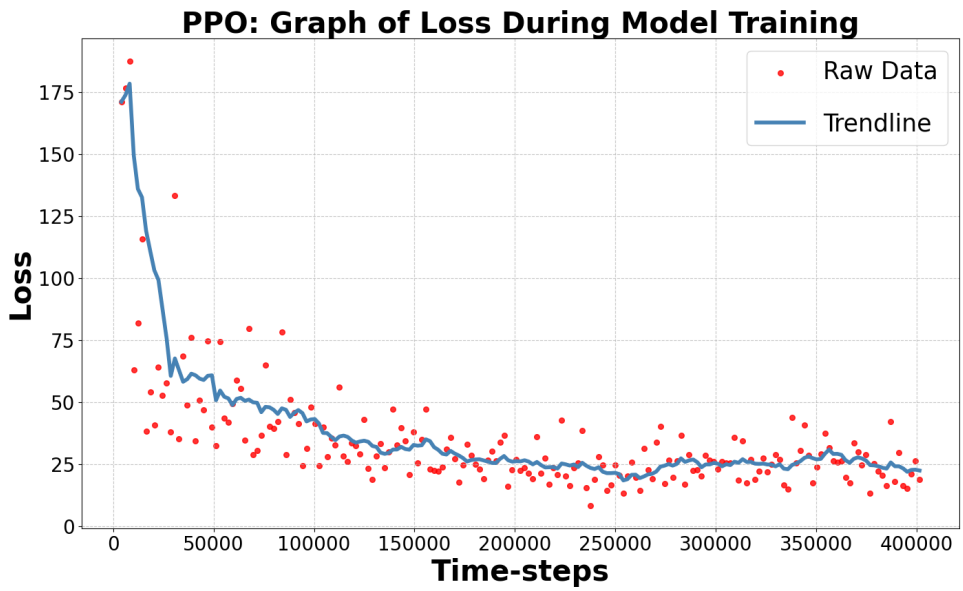


Figure 5.1: Loss Curve

Figure **??** shows loss decreasing as time-steps increase and levels out indicating that the model changes drastically at the start but gradually later in training. This can be interpreted as the agent having favourable results as it doesn't want to change too much at each time-step.

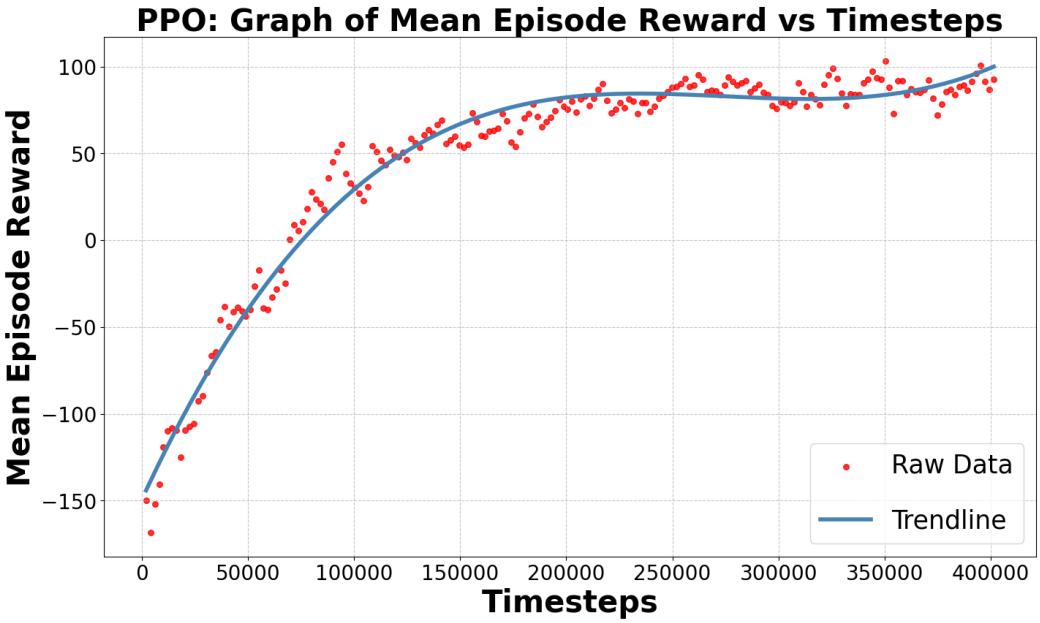


Figure 5.2: Reward Curve

Figure **??** illustrates reward increasing as the model improves at producing optimised schedules using the logic in Figure 3.

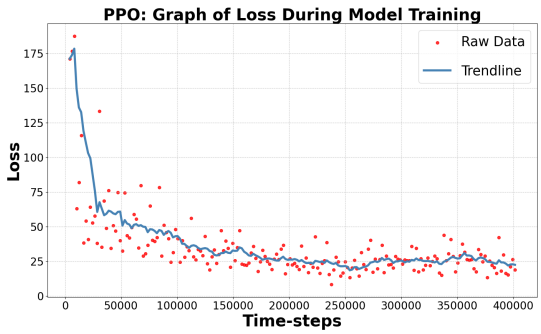


Figure 5.3: Caption

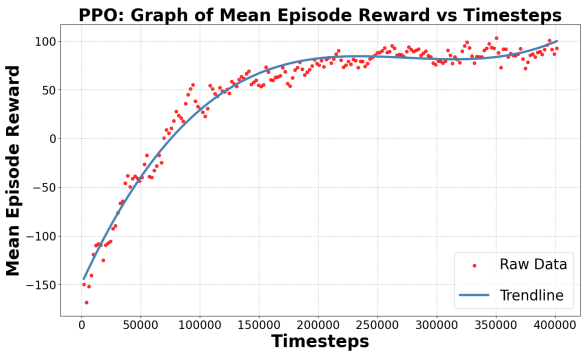


Figure 5.4: Caption

Schedule Produced

5.2 Simplified Flexible Job Shop with Buffers

Inputs

Training Curves

Schedule Produced

5.3 Simplified Flexible Job Shop with Buffers Running Shifts

Inputs

Training Curves

Schedule Produced

6 Discussion

talk about learning rates, discount factors include computer hardware specs used for training

6.0.1 Training Curves

6.0.2 Schedule Produced

6.1 Case Studies

6.1.1 Simplified Flexible Job Shop

6.1.2 Simplified Flexible Job Shop with Buffers

6.1.3 Simplified Flexible Job Shop with Buffers Running Shifts

6.2 Answers to Research Questions

6.2.1 What are the capabilities and limits of different machine learning methods, programming languages, libraries, and tools available for me to use?

The different machine learning methods have been discussed in the Literature Review under machine learning for optimisation. In the research I have shown that supervised learning is used when large amounts of data are available, unsupervised learning is used for catergorisation from large amounts of data, and reinforcement learning is used for training on simulations.
In the first section of the method, I have explained why python was chosen, stating it's capabilities and how it best fits this use case

6.2.2 What data and parameters need to be gathered, provided, and calculated for optimising manufacturing processes?

6.2.3 What programming language, machine learning method, libraries and other tools will be used in the project based on the data and parameters being used?

6.2.4 What are the capabilities and limits of this program developed?

6.2.5 Is this method of optimisation as good as or better than existing methods and how should it be deployed?

7 Environmental, Societal, and Ethical Impact

8 Conclusions

Bibliography

Ahmad, M. & Dhafr, N. (2002), 'Establishing and improving manufacturing performance measures', *Robotics and Computer-Integrated Manufacturing* **18**(3-4), 171–176.
URL:https://linkinghub.elsevier.com/retrieve/pii/S0736584502000078

Alhijawi, B. & Awajan, A. (2024), 'Genetic algorithms: theory, genetic operators, solutions, and applications', *Evolutionary Intelligence* **17**(3), 1245–1256.
URL:https://link.springer.com/10.1007/s12065-023-00822-6

Annual Financial Report 2024 (n.d.).

Babcock Annual Report and Financial Statements 2025 (n.d.).

Gao, Y.-J., Shang, Q.-X., Yang, Y.-Y., Hu, R. & Qian, B. (2023), Improved Particle Swarm Optimization Algorithm Combined with Reinforcement Learning for Solving Flexible Job Shop Scheduling Problem, *in* D.-S. Huang, P. Premaratne, B. Jin, B. Qu, K.-H. Jo & A. Hussain, eds, 'Advanced Intelligent Computing Technology and Applications', Vol. 14086, Springer Nature Singapore, Singapore, pp. 288–298. Series Title: Lecture Notes in Computer Science.
URL:https://link.springer.com/10.1007/978-981-99-4755-3$_2$5

Khuri, A. I. & Mukhopadhyay, S. (2010), 'Response surface methodology', *WIREs Computational Statistics* **2**(2), 128–149.
URL:https://wires.onlinelibrary.wiley.com/doi/10.1002/wics.73

Klar, M., Glatt, M. & Aurich, J. C. (2021), 'An implementation of a reinforcement learning based algorithm for factory layout planning', *Manufacturing Letters* **30**, 1–4.
URL:https://linkinghub.elsevier.com/retrieve/pii/S2213846321000651

Meeran, S. & Morshed, M. S. (2012), 'A hybrid genetic tabu search algorithm for solving job shop scheduling problems: a case study', *Journal of Intelligent Manufacturing* **23**(4), 1063–

1078.
URL:http://link.springer.com/10.1007/s10845-011-0520-x

Prestwich, S. (2024), 'Solving Complex Optimisation Problems by Machine Learning', *AppliedMath* **4**(3), 908–926.
URL:https://www.mdpi.com/2673-9909/4/3/49

Ragab, A., El Koujok, M., Ghezzaz, H., Amazouz, M., Ouali, M.-S. & Yacout, S. (2019), 'Deep understanding in industrial processes by complementing human expertise with interpretable patterns of machine learning', *Expert Systems with Applications* **122**, 388–405.
URL:https://linkinghub.elsevier.com/retrieve/pii/S0957417419300107

Ruiz, R. & Maroto, C. (2005), 'A comprehensive review and evaluation of permutation flowshop heuristics', *European Journal of Operational Research* **165**(2), 479–494.
URL:https://linkinghub.elsevier.com/retrieve/pii/S0377221704002553

Sabbir M (2024), 'Comparative analysis of double deep Q-network (Double DQN) and Proximal Policy Optimization (PPO) for lane-keeping in autonomous driving', *International Journal of Science and Research Archive* **13**(2), 1207–1222.
URL:https://ijsra.net/node/3887

Song, J., Wang, B. & Hao, X. (2024), 'Optimization Algorithms and Their Applications and Prospects in Manufacturing Engineering', *Materials* **17**(16), 4093.
URL:https://www.mdpi.com/1996-1944/17/16/4093

Subramaniam, V. (n.d.), 'Scheduling of Manufacturing Systems Based on Extreme Value Theory and Genetic Algorithms'.

Tang, D., Dai, M., Salido, M. A. & Giret, A. (2016), 'Energy-efficient dynamic scheduling for a flexible flow shop using an improved particle swarm optimization', *Computers in Industry* **81**, 82–95.
URL:https://linkinghub.elsevier.com/retrieve/pii/S0166361515300476

Wang, G. G. & Shan, S. (2007), 'Review of Metamodeling Techniques in Support of Engineering Design Optimization', *Journal of Mechanical Design* **129**(4), 370–380.
URL:https://asmedigitalcollection.asme.org/mechanicaldesign/article/129/4/370/466824/Review-of-Metamodeling-Techniques-in-Support-of

A Appendix A: Project Planning

B Appendix B: Risk Assessment

C Additional Appendices